
Agnostic Agent Virtual Machine (AVM)

Un Middleware Deterministico per l'Isolamento Esecutivo e l'Ottimizzazione dei Modelli Probabilistici nelle Automazioni Enterprise

Tipologia Documento: Architettura Tecnica di Riferimento (White Paper)

Stack Architetturale: PHP / Node.js (TypeScript) / Vue 3

Data di Rilascio: Luglio 2026

Stato Specifica: Definitiva / Richiesta di Implementazione

Indice Generale

Introduzione

Capitolo 1: Il Motore AST – Fondamenti Matematici e Strutture Dati

- 1.1 Fondamenti Matematici del Grafo (DAG)
- 1.2 Algoritmi di Validazione Topologica e Cycle Detection
- 1.3 Strutture Dati e Implementazione (Node.js & PHP)

Capitolo 2: JSON Mutation Protocol (JMP) – Livello di Trasporto e Schemi RPC

- 2.1 Definizione degli Schemi RPC e Primitive Atomiche
- 2.2 Il Ciclo di Parsing e Type Coercion delle Stringhe
- 2.3 Handling Deterministico degli Errori (Validation Faults)

Capitolo 3: Context Caching – Ingegneria della Memoria e Latenza Zero

- 3.1 Il Collo di Bottiglia I/O e la Meccanica della KV-Cache
- 3.2 Architettura dei Prefissi di Memoria (Prefix Caching)
- 3.3 Differential Streaming tramite State Delta

Capitolo 4: HMI State-Replication (Vue 3) – Osservabilità e Controllo Human-in-the-Loop

- 4.1 Livello di Trasporto WebSocket e Idratazione Asincrona
- 4.2 Reattività Proxy-based e Gestione dello Store Pinia
- 4.3 Ottimizzazione del Rendering di Grafi Complessi
- 4.4 Interruzione Manuale e Hot-Swapping dello Stato

Introduzione

I Large Language Models (LLM) operano su base statistico-probabilistica, calcolando la verosimiglianza del token successivo all'interno di uno spazio latente ad altissima dimensionalità. Questa natura intrinsecamente non deterministica, sebbene vantaggiosa in task creativi ed euristici, rappresenta una criticità strutturale insormontabile quando applicata all'automazione di processi enterprise rigidi o alla manipolazione diretta di infrastrutture software. I pattern tradizionali di automazione agentica (es. wrapper API o esecuzione diretta di codice generato dall'AI) espongono i sistemi a rischi critici di runtime crash, regressioni logiche e allucinazioni distruttive.

L'Agnostic Agent Virtual Machine (AVM) nasce per risolvere questa vulnerabilità architetturale. L'AVM si posiziona come un middleware intermedio (una vera e propria macchina virtuale per agenti) interposto tra l'interprete semantico probabilistico (l'LLM) e l'esecutore fisico (il runtime aziendale). Forzando l'intelligenza artificiale a operare esclusivamente tramite transizioni logiche vincolate su un albero sintattico astratto (AST) dei processi, l'AVM disinnescava le allucinazioni prima che possano toccare i sistemi di produzione. Il risultato è un framework agnostico, ultra-reattivo, reversibile e matematicamente sicuro.

Capitolo 1: Il Motore AST – Fondamenti Matematici e Strutture Dati

Il nucleo della sicurezza dell'AVM risiede nella formalizzazione dello stato dell'applicazione sotto forma di un Grafo Diretto Aciclico (DAG). L'intelligenza artificiale non scrive testo o script liberi, ma propone modifiche strutturali a questo albero sintattico astratto (AST).

1.1 Fondamenti Matematici del Grafo (DAG)

Lo stato globale del sistema S in un istante temporale t è definito univocamente dalla coppia ordinata:

$$S_t = (V_t, E_t)$$

Dove $V_t = \{v_1, v_2, \dots, v_n\}$ rappresenta l'insieme dei nodi, ciascuno dei quali mappa un'operazione operativa atomica (es. `READ_FILE`, `QUERY_DATABASE`, `EXECUTE_HTTP`). L'insieme $E_t \subseteq V_t \times V_t$ descrive gli archi orientati che esprimono le dipendenze logiche e di esecuzione: un arco diretto (v_i, v_j) impone che l'operazione v_j possa essere avviata esclusivamente in seguito al completamento con successo (stato `SUCCESS`) del nodo padre v_i .

1.1.1 La Funzione di Transizione Deterministica

L'interazione dell'LLM con il sistema si limita alla proposta di una mutazione puntuale Δ_t . La transizione verso lo stato successivo è regolata in modo deterministico dalla funzione di transizione del runtime Φ :

$$S_{t+1} = \Phi(S_t, \Delta_t)$$

Il runtime approva e applica la transizione se e solo se l'operazione soddisfa simultaneamente i vincoli di validità sintattica (V_{syn}) e integrità topologica (V_{top}):

$$V_{syn}(\Delta_t) \wedge V_{top}(S_t \cup \Delta_t) \Rightarrow True$$

In caso di violazione di uno dei due predicati, la mutazione viene rigettata immutata, azzerando l'effetto della chiamata probabilistica: $S_{t+1} = S_t$.

1.2 Algoritmi di Validazione Topologica e Cycle Detection

Per impedire stalli operativi o loop infiniti indotti dall'AI, il backend esegue una validazione preventiva su ogni mutazione strutturale.

1.2.1 Rilevamento dei Cicli (Cycle Detection)

Prima di integrare un nuovo arco nell'AST, viene eseguita un'analisi sul grafo temporaneo per escludere la presenza di cammini ciclici. L'algoritmo implementa una Ricerca in Profondità (DFS) o un tracciamento dei gradi inversi. Se l'aggiunta di una dipendenza chiude un anello relazionale, la transizione fallisce istantaneamente salvaguardando la stabilità del sistema.

1.2.2 Ordinamento Topologico (Topological Sort)

I nodi approvati in stato `PENDING` vengono schedulati per l'esecuzione reale tramite l'**Algoritmo di Kahn**. Questo processo lineare linearizza il DAG riducendo progressivamente i gradi di ingresso (in-degree) dei nodi, garantendo che l'ordine di computazione rispetti rigorosamente la precedenza delle dipendenze dei dati.

1.3 Strutture Dati e Implementazione (Node.js & PHP)

L'architettura separa le responsabilità: l'engine di validazione sintattica risiede in Node.js per via dell'efficienza nativa nel parsing degli schemi ad albero, mentre l'orchestratore di esecuzione risiede in PHP per la gestione thread-safe dei processi di backend.

1.3.1 Definizione del Nodo (Node.js / TypeScript)

```
import { z } from 'zod';

export const NodeStatusEnum = z.enum([
  'PENDING', 'VALIDATED', 'RUNNING', 'SUCCESS', 'FAILED'
]);

export const ASTNodeSchema = z.object({
  id: z.string().uuid(),
  type: z.string(),
  status: NodeStatusEnum.default('PENDING'),
  payload: z.record(z.string(), z.any()),
  dependencies: z.array(z.string().uuid()),
  metadata: z.record(z.string(), z.any()).optional()
});

export type ASTNode = z.infer<typeof ASTNodeSchema>;
```

1.3.2 L'Orchestratore di Stato (PHP)

```

<?php

class ASTOrchestrator {
    private array $nodes = [];
    private array $edges = [];

    public function applyMutation(array $mutationPayload): bool {
        // 1. Validazione Sintattica (V_syn) via IPC su microservizio Node.js
        if (!$this->validateWithNodeEngine($mutationPayload)) {
            return false;
        }

        // 2. Simulazione locale dello stato per validazione topologica (V_top)
        $tempNodes = $this->nodes;
        $tempEdges = $this->edges;

        $tempNodes[$mutationPayload['id']] = $mutationPayload;
        if (isset($mutationPayload['dependencies'])) {
            foreach ($mutationPayload['dependencies'] as $depId) {
                if (!isset($tempNodes[$depId])) return false; // Dipendenza orfana
                $tempEdges[$depId][] = $mutationPayload['id'];
            }
        }

        // 3. Cycle Detection deterministica
        if ($this->hasCycles($tempNodes, $tempEdges)) {
            return false;
        }

        // 4. Consolidamento dello stato master
        $this->nodes = $tempNodes;
        $this->edges = $tempEdges;
        return true;
    }

    private function hasCycles(array $nodes, array $edges): bool {
        $inDegree = array_fill_keys(array_keys($nodes), 0);
        foreach ($edges as $u => $neighbors) {
            foreach ($neighbors as $v) { $inDegree[$v]++; }
        }

        $queue = [];
        $visitedCount = 0;
        foreach ($inDegree as $nodeId => $degree) {
            if ($degree === 0) $queue[] = $nodeId;
        }

        while (!empty($queue)) {
            $u = array_shift($queue);
            $visitedCount++;
            if (isset($edges[$u])) {
                foreach ($edges[$u] as $v) {

```

```
        $inDegree[$v]--;  
        if ($inDegree[$v] === 0) $queue[] = $v;  
    }  
}  
}  
return $visitedCount !== count($nodes);  
}  
}
```

Capitolo 2: JSON Mutation Protocol (JMP) – Livello di Trasporto e Schemi RPC

Il protocollo JMP costituisce lo strato di serializzazione che normalizza i flussi semantici generati dall'LLM, incanalandoli all'interno di strutture dati fortemente tipizzate basate sulla specifica Remote Procedure Call (RPC).

2.1 Definizione degli Schemi RPC e Primitive Atomiche

L'LLM è confinato all'utilizzo di tre primitive fondamentali che controllano in modo esclusivo il ciclo di vita dell'AST.

2.1.1 SPAWN_NODE

Dichiara l'istanza e l'allocazione di un nodo logico nel grafo aziendale.

```
{
  "action": "SPAWN_NODE",
  "node_id": "n_5f8e2",
  "parent_id": "n_root",
  "node_type": "HTTP_REQUEST",
  "dependencies": ["n_3a1b4"]
}
```

2.1.2 MUTATE_PAYLOAD

Modifica e compila i parametri interni del payload del nodo target prima della sua esecuzione.

```
{
  "action": "MUTATE_PAYLOAD",
  "node_id": "n_5f8e2",
  "payload": {
    "method": "POST",
    "endpoint": "https://api.system.local/v1/auth",
    "timeout_ms": 5000
  }
}
```

2.1.3 YIELD_EXECUTION

Cede esplicitamente il thread di controllo al runtime, bloccando la generazione dell'AI in attesa dell'esito fisico dell'operazione.

```
{
  "action": "YIELD_EXECUTION"
}
```

2.2 Il Ciclo di Parsing e Type Coercion delle Stringhe

Durante la ricezione dell'output dello stream dell'LLM, il middleware esegue una pipeline di sanificazione a tre stadi:

- 1. Isolamento Semantico:** Il parser pulisce la stringa isolando l'oggetto JSON racchiuso nei blocchi di codice Markdown, eliminando il testo colloquiale residuo (Chain-of-Thought).
- 2. Deserializzazione Rigida:** Il payload viene decodificato. Eventuali difetti sintattici minori (es. trailing commas) vengono gestiti dal linter del protocollo o rigettati come eccezioni di parsing di basso livello.
- 3. Coercizione e Cast dei Tipi:** Il validatore esegue il type-coercion sicuro. Se l'LLM inserisce un valore numerico racchiuso tra virgolette (es. "5000") in un campo definito come integer, il runtime esegue il cast implicito previa verifica dei limiti di overflow, normalizzando l'input.

2.3 Handling Deterministico degli Errori (Validation Faults)

Quando una mutazione viola i criteri dello schema, l'AVM intercetta l'errore e istanzia un pacchetto di tipo `VALIDATION_FAULT` , impedendo l'esecuzione della pipeline di backend:

```
{
  "status": "VALIDATION_FAULT",
  "target_action": "MUTATE_PAYLOAD",
  "node_id": "n_5f8e2",
  "errors": [
    {
      "field": "payload.timeout_ms",
      "expected": "integer",
      "received": "string",
      "message": "Value must be a valid integer in milliseconds."
    }
  ]
}
```

L'architettura supporta due criteri di risoluzione configurabili: il **Self-Correction Loop**, in cui il fault viene iniettato nel contesto dell'AI per indurre una rigenerazione mirata (limitata a un massimo di 3 tentativi), e l'**Hard Stop**, che congela il thread inviando una notifica visiva all'operatore umano per una correzione manuale a tolleranza zero.

Capitolo 3: Context Caching – Ingegneria della Memoria e Latenza Zero

L'interazione continua con grafi complessi ad albero comporta una crescita lineare dei token di contesto. Senza ottimizzazioni hardware a livello di inferenza, i tempi di calcolo renderebbero l'AVM inapplicabile in contesti sincroni.

3.1 Il Collo di Bottiglia I/O e la Meccanica della KV-Cache

L'architettura dei Transformer impone che la fase di elaborazione iniziale del contesto di input (*Prefill*) presenti una complessità computazionale pari a $O(N^2)$, dove N è il numero di token. Al contrario, la generazione dei token successivi (*Decode*) è una computazione lineare a basso impatto. Reinviare l'intero albero strutturato a ogni transizione degrada il Time-To-First-Token (TTFT).

3.2 Architettura dei Prefissi di Memoria (Prefix Caching)

L'AVM azzerà il costo del prefill forzando il motore di inferenza (locale o tramite API di nuova generazione) a eseguire il **Prefix Caching** della memoria a lungo termine. Il contesto informativo viene strutturato in modo posizionale:

- 1. Segmento Statico (Cache Hit ~100%):** Include il prompt di sistema, le grammatiche EBNF e le specifiche JSON Schema del protocollo JMP. Questo blocco viene calcolato una singola volta e congelato all'interno della KV-Cache in VRAM.
- 2. Segmento Semi-Statico (Warm Cache):** Include i nodi consolidati dell'AST e la topologia storica del grafo. Viene aggiornato esclusivamente a intervalli discreti o al completamento di sotto-rami operativi.
- 3. Segmento Dinamico (Uncached Delta):** Include esclusivamente gli output reali derivati dall'esecuzione fisica dei nodi eseguiti dal runtime di backend.

3.3 Differential Streaming tramite State Delta

In seguito al comando `YIELD_EXECUTION`, l'Orchestratore non rigenera la sessione. Invia un pacchetto stringato che mappa l'esito del ciclo esecutivo:

```
{
  "event": "RUNTIME_EXECUTION",
  "results": [
    {
      "node_id": "n_5f8e2",
      "status": "SUCCESS",
      "output_summary": "HTTP 200 OK. Destination host reached."
    }
  ],
  "next_action": "AWAITING_MUTATION"
}
```

L'LLM integra nativamente questo incremento informativo sul prefisso già presente in cache, abbattendo la latenza complessiva del 90% e riducendo i costi di calcolo dei token di input a una frazione marginale.

Capitolo 4: HMI State-Replication (Vue 3) – Osservabilità e Controllo Human-in-the-Loop

La replica visiva dello stato logico della macchina virtuale è demandata alla Human-Machine Interface (HMI), un cruscotto ingegneristico progettato per garantire il monitoraggio a "scatola di vetro" (Glass-Box Observability) delle computazioni in atto.

4.1 Livello di Trasporto WebSocket e Idratazione Asincrona

La sincronizzazione dell'AST avviene in tempo reale escludendo i canali HTTP tradizionali ad alto overhead.

1. Tree Hydration: All'atto del bootstrap dell'interfaccia, il client inizializza una connessione WebSocket persistente e riceve il dump strutturato dell'intero DAG. Il payload viene serializzato tramite standard binari come MessagePack per ottimizzare i tempi di unboxing della memoria client.

2. Event Delta Streaming: Durante il funzionamento, il backend invia pacchetti di mutazione granulari contenenti esclusivamente l'ID del nodo e lo stato di transizione (es. `RUNNING` → `SUCCESS`).

4.2 Reattività Proxy-based e Gestione dello Store Pinia

Lo stato sincronizzato viene immagazzinato all'interno di un'architettura di memoria centralizzata basata su **Pinia**. Sfruttando l'implementazione dei `Proxy` nativi di Vue 3, l'interfaccia reagisce chirurgicamente alle variazioni: la mutazione del campo `status` di un singolo elemento aggiorna esclusivamente il rispettivo componente geometrico nel viewport, isolando il resto del DOM da cicli di ri-rendering parassiti.

4.3 Ottimizzazione del Rendering di Grafi Complessi

Qualora il flusso operativo generi alberi con profondità elevate o migliaia di nodi attivi, la manipolazione diretta degli elementi HTML del DOM o degli oggetti SVG puri comporta un degrado delle performance. L'HMI supera questo limite implementando un'architettura di rendering grafica astratta: la topologia globale e i vettori degli archi direzionali vengono renderizzati su un layer unico basato su **WebGL/HTML5 Canvas**, mentre i widget interattivi di modifica (form, log dettagliati)

vengono montati dinamicamente come overlay HTML condizionali esclusivamente in base alle coordinate di viewport calcolate (Virtualizzazione Spaziale).

4.4 Interruzione Manuale e Hot-Swapping dello Stato

Il monitor reattivo permette la sovrascrittura in tempo reale degli stati dei nodi aggirando le decisioni probabilistiche dell'LLM (Human-in-the-Loop esplicito).

Protocollo di Override Operatore

In presenza di un blocco logico o di un'eccezione di runtime non gestibile dall'AI, l'operatore può forzare lo stato di un nodo direttamente dall'HMI tramite l'interfaccia grafica.

L'azione manuale intercetta il nodo in stato di blocco o errore e inietta direttamente nel backend un pacchetto JMP di override:

```
{
  "action": "MUTATE_PAYLOAD",
  "node_id": "n_5f8e2",
  "payload": {
    "corrected_key": "override_value_forced_by_human"
  },
  "override_source": "HMI_OPERATOR"
}
```

L'Orchestratore PHP riceve l'input, invalida la transizione precedente e aggiorna lo stato master forzando il riavvio del flusso topologico. L'LLM acquisisce lo stato modificato dall'uomo nel ciclo di sincronizzazione successivo della KV-Cache, riprendendo il calcolo a partire dalla coordinata deterministica imposta dall'operatore.